

解题报告

姓名：白家辉

班级：计算机大类2508

学号：8208250831

日期：2026.5.17

总览

本次解题报告中，共完成 4 道题目。

题目名称	难度	知识点
P1359 租用游艇	普及	动态规划、最短路思想
P5318 查找文献	普及	图遍历、DFS、BFS
P1807 最长路	普及/提高-	DAG 最长路、动态规划、拓扑序
P5908 猫猫和企鹅	普及	树遍历、深度统计、BFS

（一）租用游艇

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1359>

知识点：动态规划、最短路思想

题目简述

长江上共有 n 个游艇出租站，游客可以在上游某站租船，并在任意下游站还船。题目给出了任意两个满足 $i < j$ 的站点之间的租金 $r_{i,j}$ 。

现在要求从 1 号站到 n 号站，计算最少需要花费多少租金。

对于全部数据， $1 \leq n \leq 200$ 。

1.00s, 128MB。

题目分析

1. 读题

虽然题目包装成“租船换乘”，但本质上是在一个编号单调递增的有向图里，从结点 1 走到结点 n ，边权是租金，要求最小总代价。

2. 性质观察

因为只能从上游去下游，所以任意转移都满足 $i < j$ 。这意味着图中不存在回边，可以直接按照站点编号从小到大进行状态转移。

如果设 $dp[j]$ 表示到达第 j 个站点的最小租金，那么最后一次租船一定是从某个 $i < j$ 的站点直接到 j ，于是有：

$$dp[j] = \min(dp[i] + r[i][j])。$$

3. 程序设计思路

当前代码更接近最短路写法：

1. 先把所有租金读入邻接矩阵 $cost$ ；
2. 用 $dist[i]$ 记录从 1 号站到 i 的当前最小花费；
3. 再配合 $visited$ 数组，反复选出当前未确定且 $dist$ 最小的站点；
4. 用这个站点去更新所有下游站点的最小花费；
5. 最终输出 $dist[n]$ 。

4. 数据结构与算法选择

虽然本题也可以直接写成 DP，但当前实现选择了“邻接矩阵 + Dijkstra 风格松弛”的方式。由于 n 只有 200 级别，这种 $O(n^2)$ 做法也完全足够，并且和“求最短路”的直觉非常一致。

5. 时空复杂度分析

- 时间复杂度： $O(n^2)$ 。当前代码每次选最小 $dist$ 结点都需要线性扫描，再做一轮线性更新。

- 空间复杂度： $O(n^2)$ 。主要来自完整保存租金矩阵 `cost`。

在 $n \leq 200$ 的范围下，这样的复杂度非常轻松。

代码实现

混合 / 评测记录 / 评测详情

R281461455 记录详情

编程语言

C O2

代码长度

1.22KB

用时

40ms

内存

1.04MB

测试点信息

源代码

测试点信息

#1	AC	#2	AC	#3	AC	#4	AC	#5	AC	#6	AC	#7	AC
3ms/812.00KB		3ms/832.00KB		3ms/812.00KB		3ms/1.04MB		3ms/788.00KB		4ms/812.00KB		3ms/824.00KB	
#8	AC	#9	AC	#10	AC	#11	AC	#12	AC				
3ms/788.00KB		3ms/812.00KB		3ms/812.00KB		4ms/812.00KB		5ms/788.00KB					

jsj0708Baijiahui

所属题目

P1359 租用游艇

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:35:47

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<limits.h>
#define MAXN 205
int cost[MAXN][MAXN];
int dist[MAXN];
int visited[MAXN];

int main()
{
    int n;
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            cost[i][j] = INT_MAX;
        }
    }
    for (int i = 1; i <= n-1; i++)
    {
        for (int j = i + 1; j <= n; j++)
        {
            scanf("%d", &cost[i][j]);
        }
    }
    for (int i = 1; i <= n; i++)
    {
        dist[i] = cost[1][i];
    }
    dist[1] = 0;
    visited[1] = 0;
    for (int i = 1; i <= n; i++)
    {
        int u = -1;
        int min_d = INT_MAX;
        for (int j = 1; j <= n; j++)
        {
            if (!visited[j] && dist[j] < min_d)
            {
                min_d = dist[j];
                u = j;
            }
        }
    }
}

```

```

    }
}
if (u == -1)break;
visited[u] = 1;
for (int v = u + 1; v <= n; v++)
{
    if (!visited[v] && cost[u][v] != INT_MAX)
    {
        if (dist[u] + cost[u][v] < dist[v])
        {
            dist[v] = dist[u] + cost[u][v];
        }
    }
}
}
printf("%d\n", dist[n]);
return 0;
}

```

(二) 查找文献

题目信息

题目链接: <https://www.luogu.com.cn/problem/P5318>

知识点: 图遍历、DFS、BFS

题目简述

有 n 篇文章和 m 条引用关系, 若存在一条边 $x \rightarrow y$, 表示文章 x 的参考文献中包含 y 。现在从 1 号文章开始阅读, 需要分别输出按规则进行 DFS 和 BFS 时的访问顺序。

如果同一时刻有多篇可以访问的文章, 必须优先访问编号更小的那一篇。

对于全部数据, $n \leq 10^5$, $m \leq 10^6$ 。

1.00s, 128MB。

题目分析

1. 读题

题目实际上就是一张有向图，从结点 1 出发做两次遍历：

1. 一次深度优先遍历；
2. 一次广度优先遍历。

需要注意的是，只遍历从 1 号文章能够到达的部分，不要求访问整张图中的所有结点。

2. 性质观察

题目要求“若有多篇可以参阅，先看编号较小的”，这意味着邻接点的访问顺序不能随意。为了让 DFS 和 BFS 都满足这个规则，通常需要先把每个结点的出边按编号从小到大排序。

3. 程序设计思路

1. 用邻接表存图；
2. 对每个结点的邻接表排序，使较小编号优先；
3. 从结点 1 出发进行 DFS，记录访问顺序；
4. 清空访问标记后，再从结点 1 出发进行 BFS，记录访问顺序；
5. 最后分别输出两种遍历结果。

当前代码里先用 `edges` 数组统计每个点的出度，再按出度动态分配每个邻接表的空间，最后通过 `qsort` 把每个结点的邻接点从小到大排序，这样 DFS 和 BFS 都能自然满足题目的访问顺序要求。

4. 数据结构与算法选择

本题使用邻接表最合适，因为边数很多。算法上分别采用 DFS 和 BFS 即可，其中：

1. DFS 负责体现“尽量往深处走”；
2. BFS 负责体现“逐层扩展”；
3. 排序保证同层或同分支下的访问次序正确。

当前实现还显式手写了 BFS 队列，并在 DFS、BFS 前后分别重置 `visited` 数组，整体实现结构非常清晰。

5. 时空复杂度分析

- 时间复杂度：遍历部分为 $O(n + m)$ ，若考虑排序，总复杂度可记为 $O(m \log m)$ 级别。

- 空间复杂度： $O(n + m)$ ，需要邻接表和访问标记。

面对 10^6 级别的边数，邻接表是必要选择。

代码实现

首页 / 评测记录 / 评测详情

R281462141 记录详情

编程语言C O2

代码长度2.05KB

用时368ms

内存11.50MB

测试点信息源代码

测试点信息

#1	#2	#3	#4	#5
AC	AC	AC	AC	AC
4ms/1.02MB	102ms/11.50MB	76ms/10.81MB	81ms/8.92MB	105ms/10.92MB

jsj0708Baijiahui

所属题目P5318 【深基18.例3】查找文献

评测状态Accepted

提交时间2026-06-11 17:41:52

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 100005
#define MAXM 1000005

typedef struct Edge
{
    int u, v;
} Edge;

Edge edges[MAXM];
int *adj[MAXN];
int degree[MAXN];
int visited[MAXN];
int n, m;

int cmp(const void *a, const void *b)
{
    return *(int *)a - *(int *)b;
}

void dfs(int u)
{
    visited[u] = 1;
    printf("%d ", u);
    for (int i = 0; i < degree[u]; i++)
    {
        int v = adj[u][i];
        if (!visited[v])
        {
            dfs(v);
        }
    }
}

void bfs(int start)
{
    memset(visited, 0, sizeof(visited));
    int *queue = (int *)malloc((n + 1) * sizeof(int));
    int front = 0, rear = 0;
    queue[rear++] = start;

```



```

visited[start] = 1;
printf("%d ", start);
while (front < rear)
{
    int u = queue[front++];
    for (int i = 0; i < degree[u]; i++)
    {
        int v = adj[u][i];
        if (!visited[v])
        {
            visited[v] = 1;
            printf("%d ", v);
            queue[rear++] = v;
        }
    }
}
free(queue);
}

int main()
{
    scanf("%d %d", &n, &m);

    for (int i = 0; i < m; i++)
    {
        int u, v;
        scanf("%d %d", &u, &v);
        edges[i].u = u;
        edges[i].v = v;
        degree[u]++;
    }

    for (int i = 1; i <= n; i++)
    {
        if (degree[i] > 0)
        {
            adj[i] = (int *)malloc(degree[i] * sizeof(int));
        } else
        {
            adj[i] = NULL;
        }
    }
}

```

```

memset(degree, 0, sizeof(degree));
for (int i = 0; i < m; i++)
{
    int u = edges[i].u;
    int v = edges[i].v;
    adj[u][degree[u]++] = v;
}

for (int i = 1; i <= n; i++)
{
    if (degree[i] > 0)
    {
        qsort(adj[i], degree[i], sizeof(int), cmp);
    }
}

memset(visited, 0, sizeof(visited));
dfs(1);
printf("\n");

bfs(1);
printf("\n");
for (int i = 1; i <= n; i++)
{
    if (adj[i] != NULL)
    {
        free(adj[i]);
    }
}
return 0;
}

```

(三) 最长路

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1807>

知识点：DAG 最长路、动态规划、拓扑序

题目简述

给定一个带权有向无环图，顶点编号为 1 到 n ，边满足 $u < v$ 。要求求出从 1 到 n 的最长路径长度。

若 1 无法到达 n ，则输出 -1 。

对于全部数据， $n \leq 1500$ ， $m \leq 5 \times 10^4$ ，边权允许为负数。

1.00s, 128MB。

题目分析

1. 读题

这道题要求的是最长路，而不是最短路。若图中允许有环，最长路通常会比较麻烦，但题目已经说明这是有向无环图，因此问题会简单很多。

2. 性质观察

由于所有边都满足 $u < v$ ，顶点编号本身就是一个合法的拓扑序。这样就不需要额外做拓扑排序，直接按 1 到 n 的顺序做状态转移即可。

设 $dp[v]$ 表示从 1 走到 v 的最长路径长度，那么对于每条边 $u \rightarrow v$ ，都可以尝试用：

$$dp[v] = \max(dp[v], dp[u] + w)。$$

需要注意的是，边权可以为负，因此不能把未到达状态初始化为 0 ，而应初始化为负无穷。

3. 程序设计思路

1. 用邻接表存储所有边；
2. 初始化 $dp[1]=0$ ，其余为负无穷；
3. 按顶点编号从小到大枚举每个 u ；
4. 若 $dp[u]$ 有效，则遍历它的所有出边，更新终点的最长路值；
5. 最后若 $dp[n]$ 仍为负无穷，说明无法到达，输出 -1 ，否则输出 $dp[n]$ 。

当前代码把这个 dp 数组命名为 $dist$ ，并用链式前向风格的邻接表存图；由于顶点编号本身就是拓扑序，所以直接按 $u=1..n$ 扫描并松弛即可。

4. 数据结构与算法选择

本题适合使用“拓扑序上的动态规划”。当前实现配合链式邻接表完成转移，既节省空间，也让每条边只会被顺序访问一次。

5. 时空复杂度分析

- 时间复杂度： $O(n + m)$ ，每条边只会被扫描一次。
- 空间复杂度： $O(n + m)$ ，需要邻接表和 DP 数组。

这完全能够满足题目的数据范围。

代码实现

洛谷 / 评测记录 / 评测详情

R281462363 记录详情

编程语言C O2 | 代码长度886B | 用时76ms | 内存1.96MB

测试点信息 | 源代码

测试点信息

#1 AC 16ms/1.96MB	#2 AC 14ms/1.90MB	#3 AC 4ms/788.00KB	#4 AC 4ms/812.00KB	#5 AC 4ms/808.00KB	#6 AC 4ms/636.00KB	#7 AC 5ms/812.00KB
#8 AC 14ms/1.87MB	#9 AC 4ms/816.00KB	#10 AC 3ms/788.00KB	#11 AC 4ms/812.00KB			

jsj0708Baijiahui

所属题目P1807 最长路

评测状态Accepted

评测分数100

提交时间2026-06-11 17:43:55

```

#include<stdio.h>
#include<stdlib.h>
#include<limits.h>
typedef struct Edge
{
    int to;
    int weight;
    struct Edge* next;
}Edge;
Edge* adj[1505];
long long dist[1505];
const long long INF = 1e18;
void addEdge(int u, int v, int w)
{
    Edge* e = (Edge*)malloc(sizeof(Edge));
    e->to = v;
    e->weight = w;
    e->next = adj[u];
    adj[u] = e;
}
int main()
{
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++)
    {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        addEdge(u, v, w);
    }
    for (int i = 1; i <= n; i++)
    {
        dist[i] = -INF;
    }
    dist[1] = 0;
    for (int u = 1; u <= n; u++)
    {
        if (dist[u] == -INF)continue;

        for (Edge* e = adj[u]; e != NULL; e = e->next)
        {
            int v = e->to;
            int w = e->weight;

```

```

        if (dist[v] < dist[u] + w)
        {
            dist[v] = dist[u] + w;
        }
    }
}
if (dist[n] == -INF)
{
    printf("-1\n");
}
else
{
    printf("%lld\n", dist[n]);
}
return 0;
}

```

(四) 猫猫和企鹅

题目信息

题目链接：<https://www.luogu.com.cn/problem/P5908>

知识点：树遍历、深度统计、BFS

题目简述

有 n 个居住区和 $n - 1$ 条道路，保证整张图连通，因此它本质上是一棵树。除 1 号居住区外，每个居住区住着一只小企鹅。

一只猫猫从 1 号居住区出发，只愿意拜访距离自己不超过 d 的小企鹅。要求输出它最多能拜访多少只。

对于全部数据， $1 \leq n, d \leq 10^5$ 。

1.00s, 128MB。

题目分析

1. 读题

题目中的“道路长度都为 1，整张图连通且边数为 $n-1$ ”说明这是一个无权树。问题就转化成：求整棵树中，有多少个结点到根结点 1 的距离不超过 d 。

2. 性质观察

在树中，从根到任意结点的路径唯一，所以“距离”实际上就是该结点的深度。于是只要统计深度不超过 d 的结点数即可。

但要注意 1 号居住区没有企鹅，因此即使它的深度为 0，也不应被计入答案。

3. 程序设计思路

1. 用邻接表建树；
2. 从结点 1 开始做一次 BFS 或 DFS；
3. 在遍历过程中记录每个结点到根的距离；
4. 若某个结点的距离 $\leq d$ 且它不是根结点，就把答案加一；
5. 所有结点处理完成后输出答案。

当前代码实际采用的是 BFS：先手工建立无向邻接表，再用数组队列从 1 号结点逐层扩展，同时记录每个点的 `depth`，最后顺序统计 $2..n$ 中深度不超过 d 的结点数。

4. 数据结构与算法选择

本题适合使用邻接表配合 BFS。因为边权全为 1，BFS 天然可以按层扩展，第一层对应距离 1，第二层对应距离 2，统计起来很自然。

5. 时空复杂度分析

- 时间复杂度： $O(n)$ ，整棵树的每条边和每个结点最多访问一次。
- 空间复杂度： $O(n)$ ，需要邻接表、队列和访问标记。

在线性复杂度下可以稳妥通过 10^5 规模的数据。

代码实现

首页 / 评测记录 / 评测详情

R281461349 记录详情

编程语言

C O2

代码长度

1.25KB

用时

156ms

内存

8.41MB

测试点信息

源代码

测试点信息

#1	#2	#3	#4	#5	#6
AC	AC	AC	AC	AC	AC
31ms/8.25MB	32ms/8.41MB	32ms/8.37MB	32ms/8.41MB	26ms/8.27MB	3ms/800.00KB

jsj0708Baijiahui

所属题目

P5908 猫猫和企鹅

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:34:56


```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#define MAXN 100001
typedef struct AdjNode
{
    int v;
    struct AdjNode* next;
}AdjNode;
AdjNode* adj[MAXN];
int depth[MAXN];
int visited[MAXN];

int main()
{
    int n, d;
    scanf("%d %d", &n, &d);
    for (int i = 1; i <= n ; i++)
    {
        adj[i] = NULL;
        visited[i] = 0;
        depth[i] = 0;
    }
    for (int i = 0; i < n - 1; i++)
    {
        int u, v;
        scanf("%d %d", &u, &v);
        AdjNode* node1 = (AdjNode*)malloc(sizeof(AdjNode));
        node1->v = v;
        node1->next = adj[u];
        adj[u] = node1;

        AdjNode* node2 = (AdjNode*)malloc(sizeof(AdjNode));
        node2->v = u;
        node2->next = adj[v];
        adj[v] = node2;
    }

    int* queue = (int*)malloc((n + 1) * sizeof(int));
    int front = 0;
    int rear = 0;
    queue[rear++] = 1;
    visited[1] = 1;

```

```

depth[1] = 0;
while (front < rear)
{
    int u = queue[front++];
    AdjNode* p = adj[u];
    while (p != NULL)
    {
        int v = p->v;
        if (!visited[v])
        {
            visited[v] = 1;
            depth[v] = depth[u] + 1;
            queue[rear++] = v;
        }
        p = p->next;
    }
}
int cnt = 0;
for (int i = 2; i <= n; i++)
{
    if (depth[i] <= d)
        cnt++;
}
printf("%d\n", cnt);
free(queue);
for (int i = 1; i <= n; i++)
{
    AdjNode* p = adj[i];
    while (p != NULL)
    {
        AdjNode* tmp = p;
        p = p->next;
        free(tmp);
    }
}
return 0;
}

```